

PROJECT-2: CI/CD PIPELINE

OBJECTIVE:

The primary objective of this project is to establish a robust CI/CD pipeline that automates the process of building, testing, and deploying applications by integrating Git, Jenkins, Maven, SonarQube, Docker Hub, Ansible, Kubernetes (EKS) and Terraform, the project aims to achieve the following goals:

Project Goals

1. **Infrastructure as Code (IaC):** Use Terraform to provision the entire infrastructure, including:
 - **AWS Resources:** VPC, subnets, security groups, EC2 instances, route tables, internet gateways, and NAT gateways.
 - **CI/CD Components:** UTIL Sever (Git, Jenkins, ansible, docker, maven, sonarqube, trivy components), App server (EKS Cluster)
2. **Version Control:** Maintain application source code and infrastructure code in Git repositories to ensure version control and collaboration.
3. **Continuous Integration:**
 - **Jenkins:** Automate the build and test processes using Jenkins pipelines.
 - **Maven:** Build and manage dependencies of Java applications.
4. **Static Code Analysis:**
 - **SonarQube:** Perform static code analysis to ensure code quality and adherence to standards.
5. **Containerization:**
 - **Docker:** Containerize applications for consistent deployment environments.
 - **Docker Hub:** Store and manage Docker images.
 - **Trivy (docker image scan):** docker image scanner
6. **Continuous Deployment:**
 - **Ansible:** For continuous deployment to Kubernetes (EKS) cluster.
 - **Kubernetes (EKS):** Deploy and manage containerized applications using Amazon EKS.

SUMMARY:

This end-to-end Jenkins pipeline will automate the entire CI/CD process for a Java Spring application, from code checkout to production deployment, using popular devops tools like Git, Jenkins, Maven, SonarQube, Docker Hub, Trivy, Ansible, Kubernetes and Terraform.

This project covers setting up a CI/CD pipeline using.

- Infrastructure as Code: Provision infrastructure with Terraform (VPC, subnets, security groups, EC2 instances(UTIL Sever (Git, Jenkins, ansible, docker, maven, sonarqube, trivy components), App server (EKS Cluster))).
- Configuring AWS CLI, installing kubectl and eksctl in app server.
- Creating an EKS cluster with eksctl.
- Installing necessary Jenkins plugins and setting up credentials.

Creating a Jenkins pipeline to:

- Checkout code from GitHub.
- Perform a SonarQube scan.
- Build the project using Maven.
- Build Docker Image using Docker file and push Docker images to Docker Hub.
- Trivy (docker image scan): docker image scanner
- Update deployment configurations and push them to a Git repository.
- Deploy the application to Kubernetes using Ansible.

Pre-requisites:

1. Java application code hosted on a Git repository

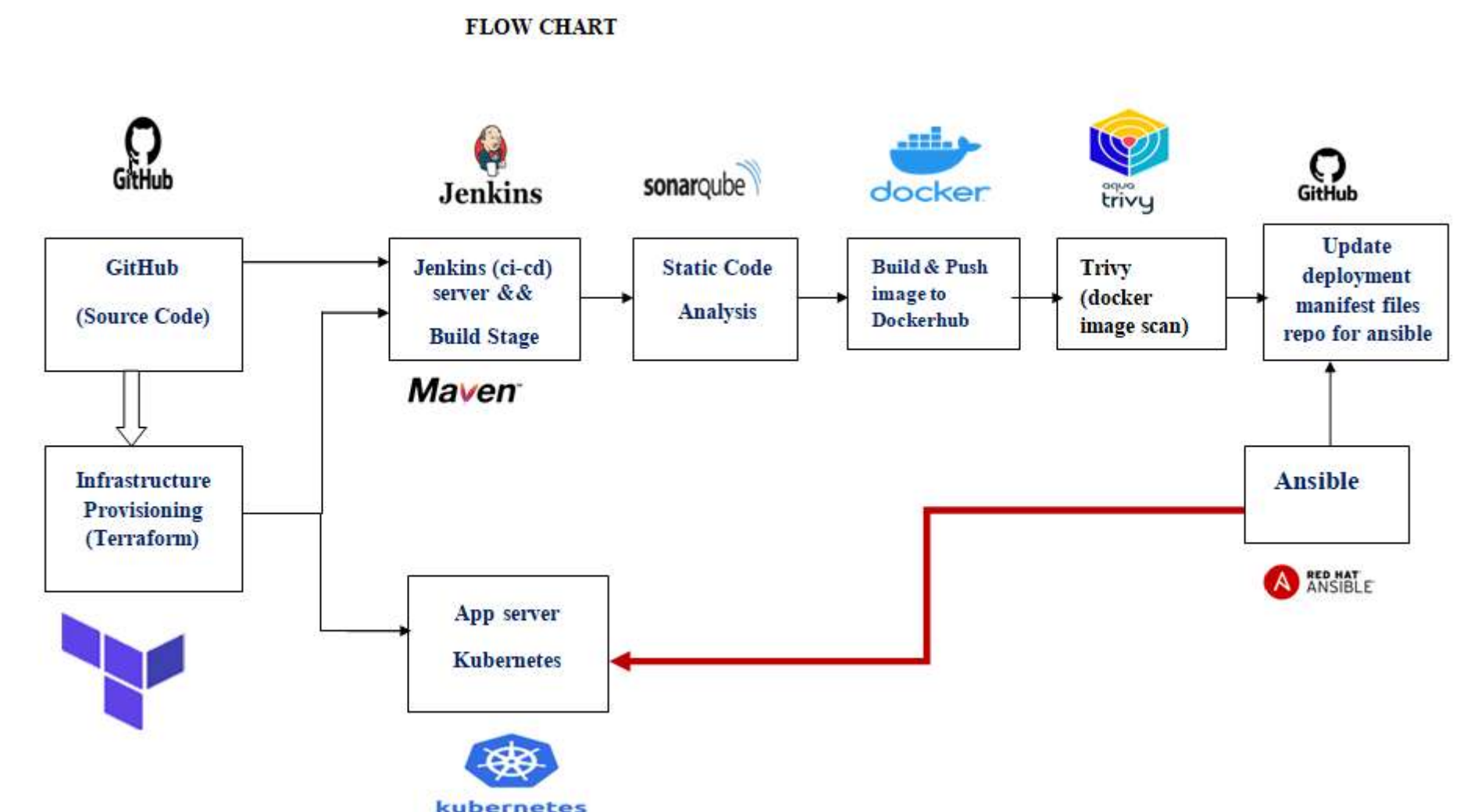
Account and Access

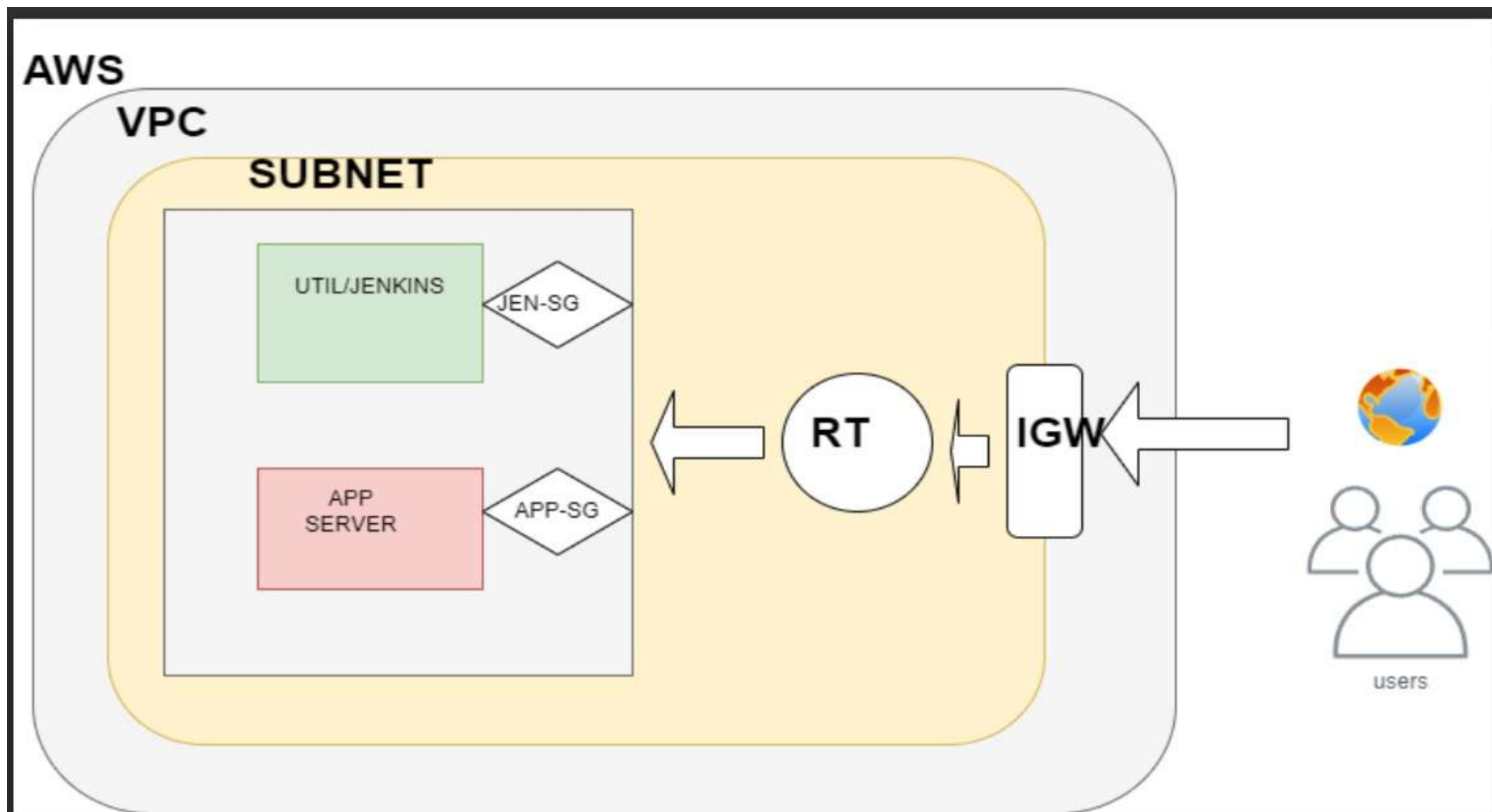
1. **AWS Account:** AWS account with necessary permissions to create resources.

Tools and Services

1. **Terraform:** Infrastructure as Code tool for provisioning AWS resources.
2. **Terraform Scripts:** Scripts to create the necessary AWS infrastructure (VPC, subnets, security groups, EC2 instances (UTIL Sever (Git, Jenkins , ansible , docker, maven ,sonarqube, trivy components) , App server (EKS Cluster))).

FLOW CHART:





1. GitHub (Source Code):

- **Function:** GitHub serves as the version control system where the source code of the application is stored and managed.
- **Flow:** Developers push their code changes to the GitHub repository.

Github url: <https://github.com/devopstraininghub/REALTIMEPROJECT.git>

2. Infrastructure Provisioning (Terraform):

- **Function:** Terraform is used to define and provision the complete infrastructure required for the application, such as VPCs, subnets, security groups, EC2 instances.

3. Jenkins (CI/CD Server & Build Stage):

- **Function:** Jenkins automates the continuous integration and continuous delivery (CI/CD) pipeline. It pulls the latest code from GitHub, builds the application using Maven, runs tests, and performs static code analysis.
- **Flow:** Jenkins fetches the source code from GitHub, compiles it, and moves to the next stage.

4. Maven:

- **Function:** Maven is used for building Java applications. It manages project dependencies, compiles the source code, runs tests, and packages the application.
- **Flow:** Maven builds the application as part of the Jenkins pipeline.

5. SonarQube (Static Code Analysis):

- **Function:** SonarQube analyzes the source code to identify bugs, vulnerabilities, and code smells.
- **Flow:** Jenkins triggers a SonarQube scan as part of the CI process. The results are reviewed to ensure code quality before moving to the next stage.

6. Docker (Build & Push Image to DockerHub):

- **Function:** Docker containerizes the application, creating a portable image that can be run consistently across different environments.
- **Flow:** Jenkins builds a Docker image of the application and pushes it to DockerHub for storage and versioning.

7. Trivy (Docker Image Scan):

- **Function:** Trivy is a vulnerability scanner for container images. It scans Docker images for security vulnerabilities before deployment.
- **Flow:** Jenkins uses Trivy to scan the Docker image. If vulnerabilities are found, they must be addressed before deployment.

8. GitHub (Update Deployment Manifest Files Repo for Ansible):

- **Function:** This repository contains Kubernetes deployment manifests and other configuration files needed for deployment.
- **Flow:** Jenkins updates Kubernetes deployment manifests .

9. Ansible:

- **Function:** Ansible is used for application deployment.
- **Flow:** Ansible pulls the latest deployment manifests from GitHub and uses them to deploy the application to the Kubernetes cluster.

10. App Server (Kubernetes):

- **Function:** Kubernetes manages the deployment, scaling, and operation of containerized applications. EKS (Elastic Kubernetes Service) provides a managed Kubernetes cluster on AWS.
- **Flow:** Ansible deploys the application to the Kubernetes cluster. The application is now running and managed by Kubernetes.

❖ Flow Summary

- ✓ Developers push code to GitHub.
- ✓ Terraform provisions the necessary infrastructure.
- ✓ Jenkins pulls code from GitHub, builds it with Maven, runs static code analysis with SonarQube, builds a Docker image, and pushes it to DockerHub.
- ✓ Trivy scans the Docker image for vulnerabilities.
- ✓ Jenkins updates the deployment manifests in the GitHub repository for Ansible.
- ✓ Ansible deploys the application to the Kubernetes cluster on EKS.

Installations and Setting up:

➤ Github url: <https://github.com/devopstraininghub/REALTIMEPROJECT.git>

➤ Terraform installation on windows:

https://releases.hashicorp.com/terraform/1.8.5/terraform_1.8.5_windows_386.zip

- Download the Terraform ZIP file from the link above.
- Extract the ZIP file to a directory (e.g., C:\terraform).
- Add the Terraform directory to the system's PATH environment variable

➤ AWS CLI Configuration on Windows

- **Steps:**

- Install AWS CLI if not already installed:

- Download the installer from [AWS CLI Installer](#).
- Run the installer and follow the on-screen instructions.

- Configure AWS CLI:

aws configure

- This will prompt for AWS Access Key ID, Secret Access Key, Default Region, and Default Output Format.

➤ Setting Up the CI/CD Pipeline

a. Infrastructure Provisioning with Terraform

- **Steps:**

1. Define Terraform scripts to create VPC, subnets, security groups, EC2 instances(Jenkins and app server),and other necessary AWS resources.

2. Initialize Terraform:

terraform init

3. Plan the infrastructure changes:

terraform plan

4. Apply the changes to deploy the infrastructure:

terraform apply

b. Version Control with Git

- **Steps:**

1. Maintain separate repositories for application source code and infrastructure code.

2. Generate a GitHub token for Jenkins to access your GIT repositories:

- Go to GitHub > Settings > Developer settings > Personal access tokens.

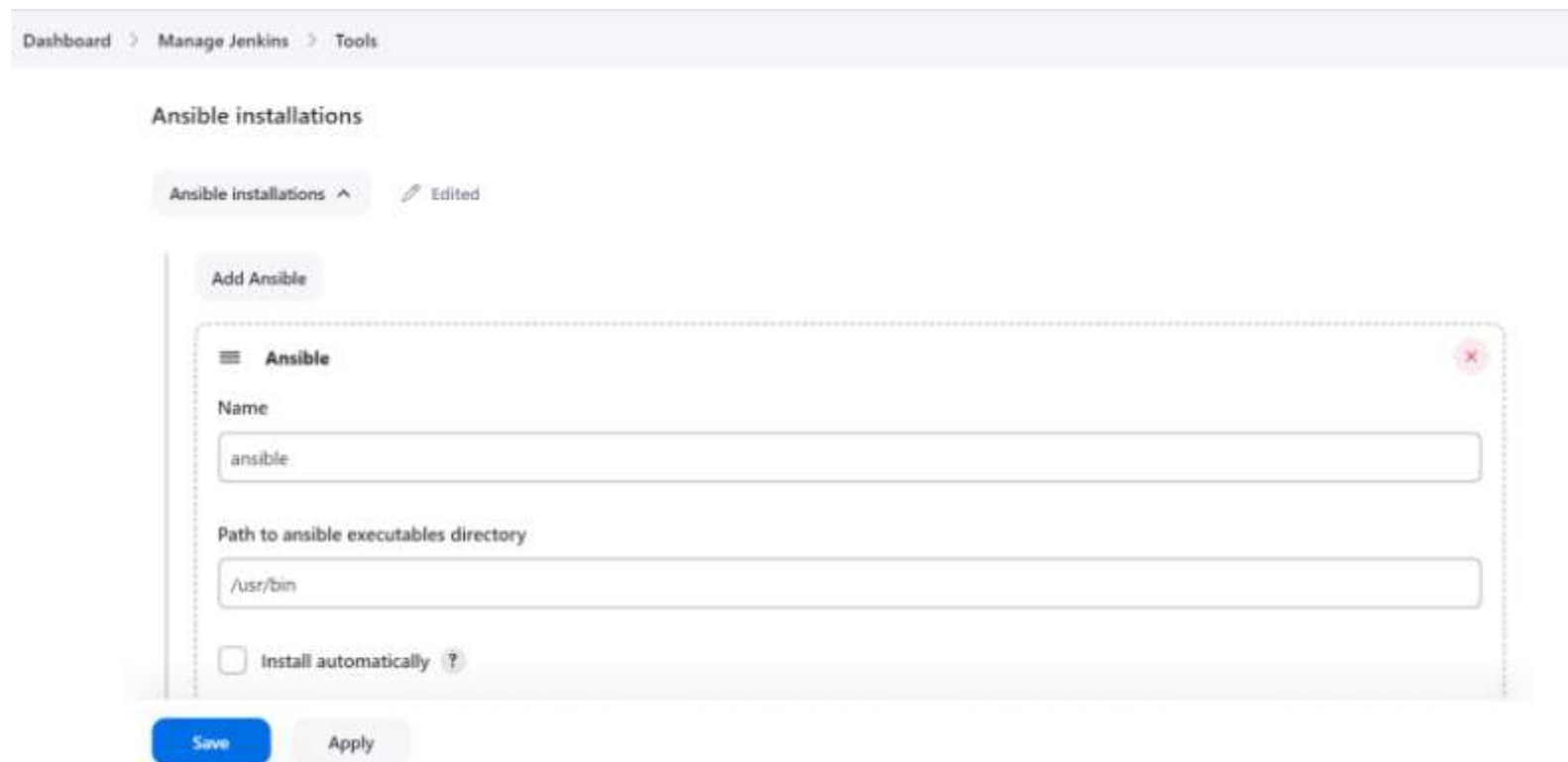
3. Add github token to Jenkins global credentials as a secret txt

c. CI/CD Pipeline with Jenkins

- **Steps:**

1. Set up Jenkins on an EC2 instance provisioned by Terraform.
2. Create Jenkins pipelines to automate the build and test processes.
3. Install plugins: ansible,pipeline stage view

4. Add ansible installation in tools



5. Integrate Maven for building Java applications and SonarQube for static code analysis.

d. Static Code Analysis with SonarQube

1. Integrate SonarQube with Jenkins to run static code analysis as part of the CI pipeline.

e. Containerization and Docker Hub

1. Add docker hub password to Jenkins credentials as a secret txt
2. Push Docker images to Docker Hub for centralized storage and versioning.

f. Continuous Deployment with Ansible

1. Generate ssh-keygen in Jenkins server as a ec2-user
2. Add this public key in app server(ec2-user) path=> .ssh/authorized_keys
3. Add app server public ip in Jenkins server path=> /etc/ansible/hosts
4. Check the connection with app server in Jenkins server command

ansible -i /etc/ansible/hosts -m ping all

5. Install eksctl,kubectl in app server (kubernetes)

kubectl installation on aws linux machine

- curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
- curl -LO "https://dl.k8s.io/release/\$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl.sha256"
- sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl

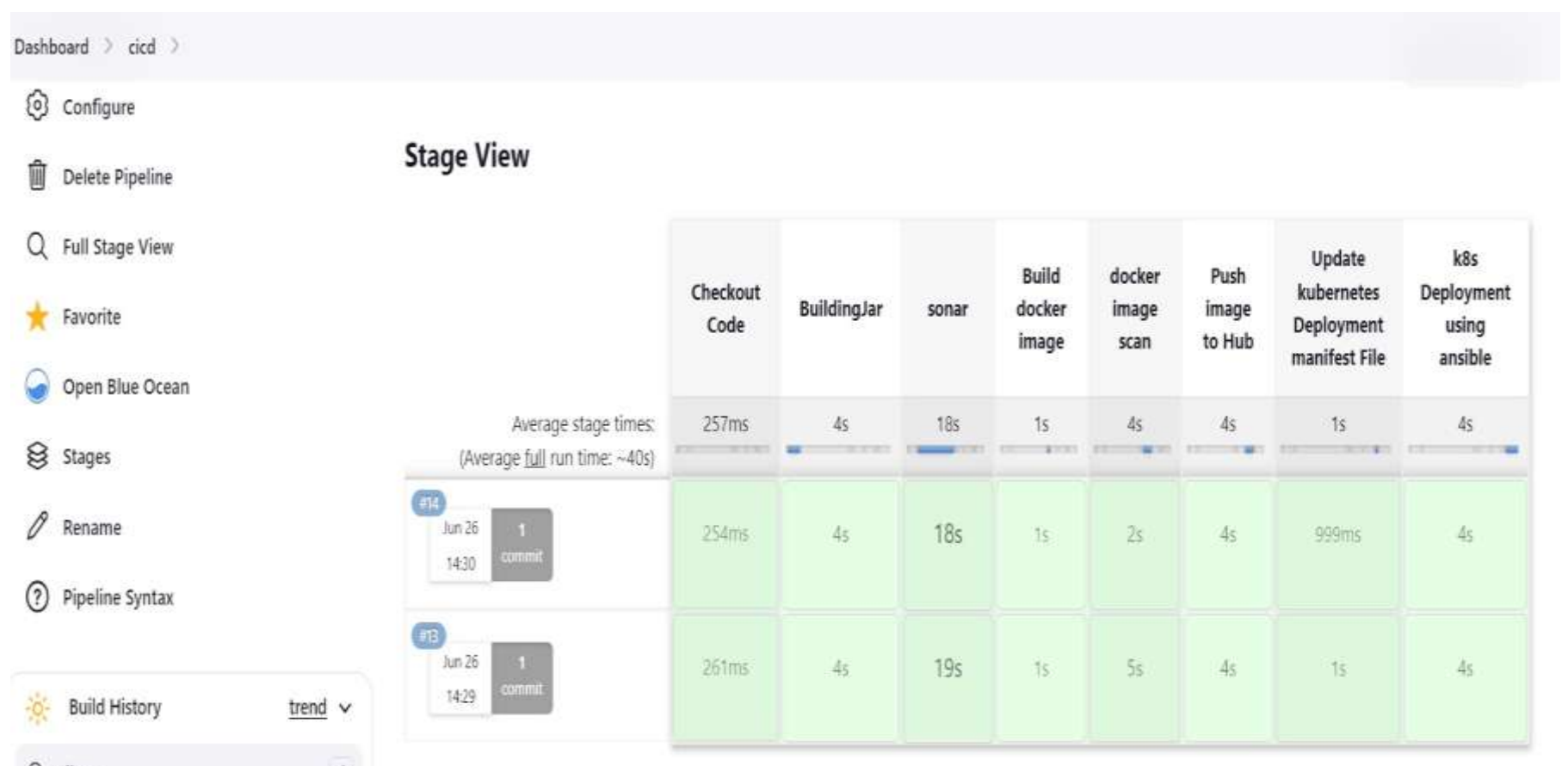
eksctl installation on aws linux machine

- `curl --silent --location "https://github.com/weaveworks/eksctl/releases/latest/download/eksctl_$(uname -s)_amd64.tar.gz" | tar xz -C /tmp`
- `sudo mv /tmp/eksctl /usr/local/bin`

6. Use Ansible to manage Kubernetes manifests stored in GitHub repositories.

Github url: <https://github.com/devopstraininghub/REALTIMEPROJECT.git>

7. Deploy applications to the EKS cluster using Ansible playbook.



```
ec2-user@app ~]$ sudo kubectl get all
NAME                                READY    STATUS    RESTARTS   AGE
pod/mc-app-7797fc7bf4-89xzt         1/1     Running   0           18s
pod/mc-app-7797fc7bf4-s5pk7         1/1     Running   0           18s

NAME                                TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)
service/kubernetes                  ClusterIP      10.100.0.1     <none>          443/TCP
service/mc-app-service              LoadBalancer  10.100.226.223 a2a5e5f083b26430fadf920fe4f2a782-1459326598.us-east-1.elb.amazonaws.com 80:3449/TCP 4h19m

NAME                                READY    UP-TO-DATE    AVAILABLE    AGE
deployment.apps/mc-app              2/2      2              2             19s

NAME                                DESIRED    CURRENT    READY    AGE
replicaset.apps/mc-app-7797fc7bf4  2          2          2        19s
ec2-user@app ~]$ |
```

Paste it on your browser and see your application :

Loadbalancer url : **a2a5e5f083b26430fadf920fe4f2a782-1459326598.us-east-1.elb.amazonaws.com**